# A Deep Learning Framework for classifying Hog Plum leaf diseases

Anika Nawal: 2021-3-60-161

Md. Asifuzzaman: 2021-3-60-167

Tawhidur Rahman: 2021-3-60-233

Abdul Muhaimin Ibne Ali: 2021-3-60-172

Md. Touhidur Rahman Limon: 2022-1-60-272

## Introduction:

The proposed methodology starts with preprocessing the data from the plum leaf dataset through a series of image enhancement steps including noise removal, contrast enhancement, smoothing, and resizing. These preprocessing techniques prepare the images for more effective analysis. Following this, data augmentation techniques such as rotation, flipping, and color jitter are applied to increase the variety and generalizability of the dataset. Once the dataset is augmented, feature selection is conducted using Recursive Feature Elimination (RFE) to identify the most relevant and informative features for training. Subsequently, multiple deep learning models - namely **ConvNeXt**, **RegNet**, **ResNeXt-50**, **DenseNet-201**, **EfficientNetV2-S**, and a custom **NASNet-Large** are trained on the enhanced dataset.

The performance of these models is evaluated using a comprehensive set of metrics, including accuracy, precision, recall, F1-score, area under the ROC curve (AUC), confusion matrix, and learning curves. Finally, to ensure interpretability and to highlight the regions of interest in the leaf images, Grad-CAM is employed. This technique visualizes the areas the models focus on when predicting disease, thereby enhancing the transparency, effectiveness, and robustness of the proposed disease detection system.

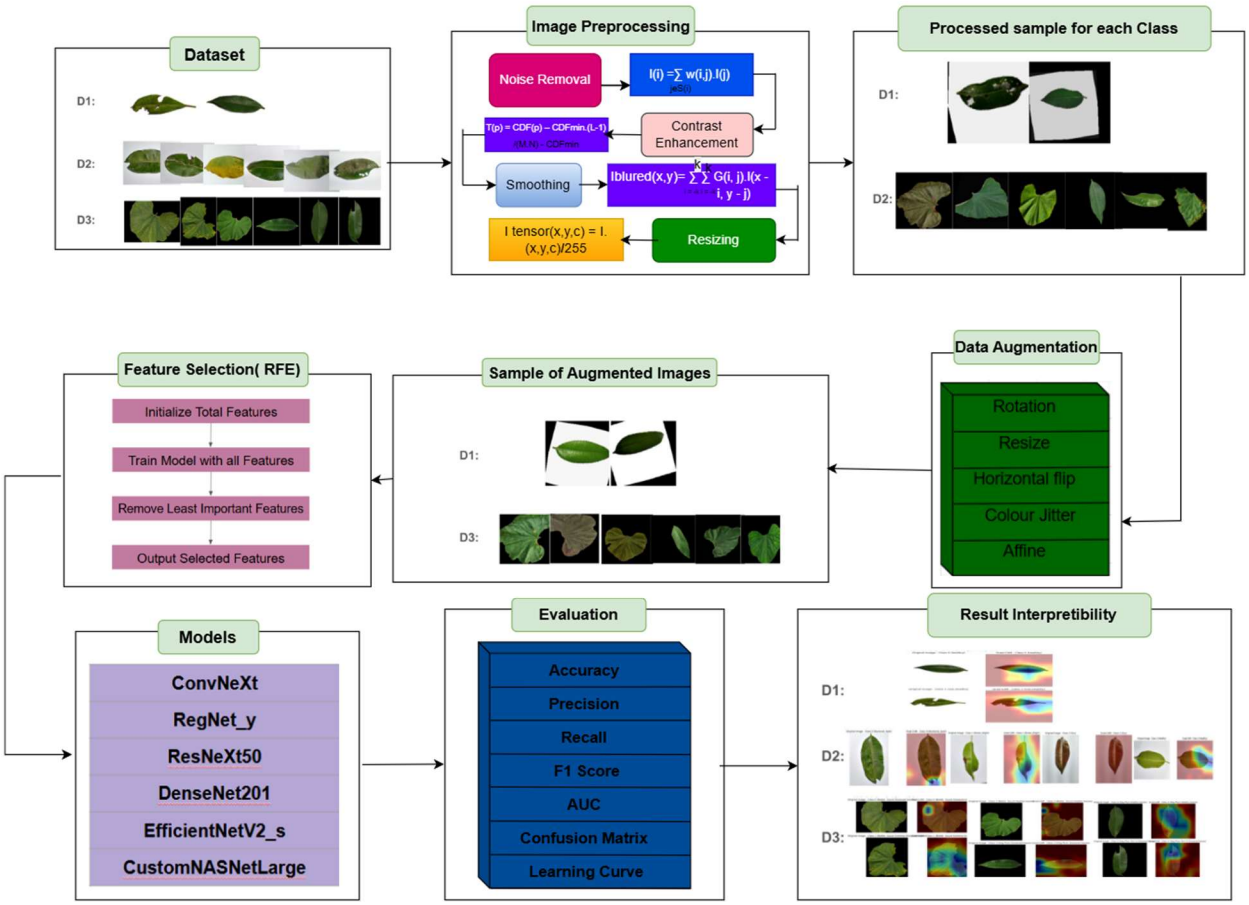


**Fig-1:** Proposed Methodology

# 1. Methodology:

## 1.1 Data Description

**Table 1:** Category wise Distribution of Plum Leaf Images

| **Disease category** | **Number of Images** |
|---|---|
| Healthy | 2829 |
| Not-healthy | 953 |
| **Total** | 3782 |

**Fig-2:** Representative samples from the Hog Plum leaf diseases Dataset

Here The Dataset is Hog Plum leaf diseases which was developed by researchers to support the ML-based classification of Plum leaf diseases and to aid in plant health monitoring. It contains 3782 processed images organized into two categories that include One is healthy and another is Not Healthy. In Healthy leaf diseases there are 2829 images and in Not Healthy diseases there are 953 images. Here the Figure 2 we see a sample image of those class. The dataset was found in Captured across multiple locations in Bangladesh with field surveys guided by experts. To enhance the dataset size for DL applications, data augmentation techniques were employed, bringing the total number of images to 5657 which has a Healthy dataset has 2829 images and not healthy has 2828 images. This dataset is especially valuable for researchers and agricultural experts developing models for automated plant disease classification.

### 1.1.1 Data Augmentation Technique

**Table 2:** Number of Images for Each Class After Performing Data Augmentation (Training 80%, Validation 5%, Testing 15%)

| Class | Original Count | Target Count | Training (80%) | Validation (5%) | Test (15%) |
|---|---|---|---|---|---|
| Healthy | 2829 | 2829 | 2263 | 141 | 424 |
| Not-healthy | 953 | 2828 | 2262 | 141 | 424 |
| **Total** | 3782 | 5657 | 4525 | 282 | 848 |

### 1.1.2 Class Distribution dataset

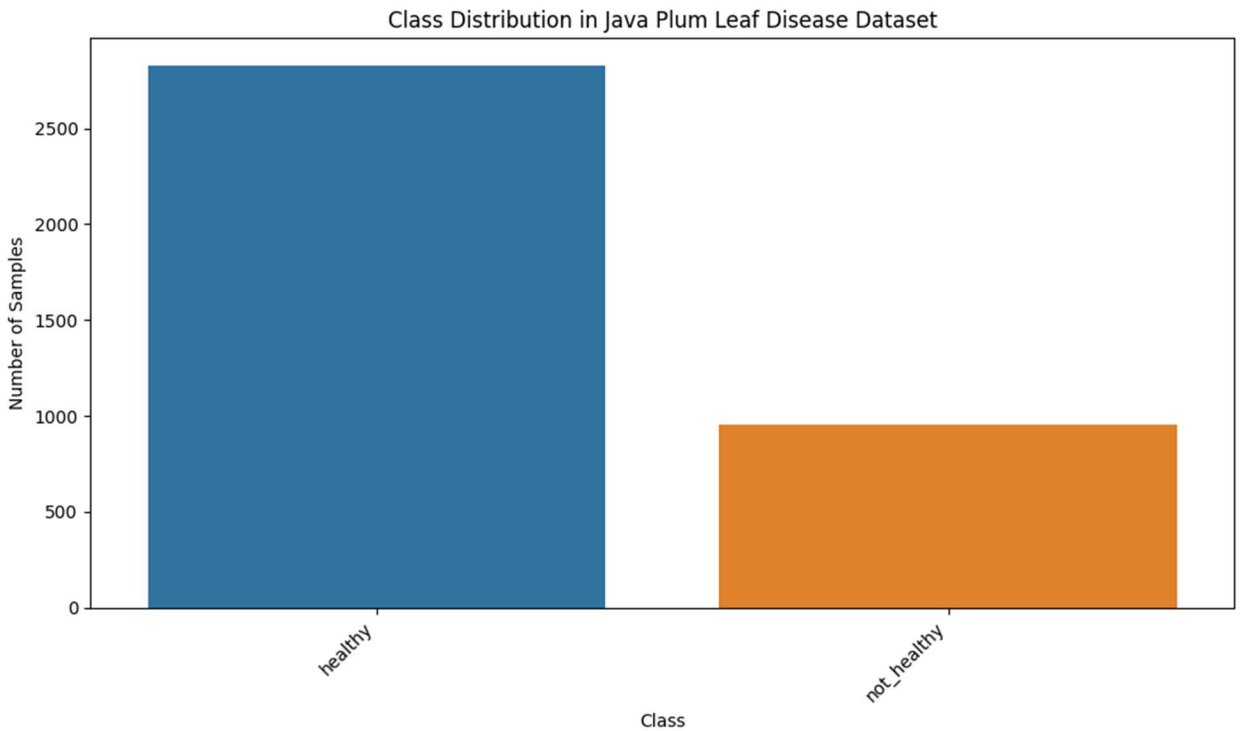


**Fig-3:** Class distribution of experimental dataset

From Figure 3 we observed that the dataset exhibits a class-imbalanced distribution, we see healthy class have 2829 images and not healthy class which has 953 images.

## 1.2 Model Architecture

### 1.2.1 ConvNeXt

ConvNeXt is a modern convolutional neural network designed to be as effective as vision transformers but with CNN efficacy. The network starts with a 4×4 stem convolution reducing the input image (224×224×3) to 56×56×96. There are four phases in the model, and each phase is made up of a sequence of ConvNeXt blocks. Each block has a 7×7 depth wise convolution, channels last LayerNorm, a pointwise convolution for expansion by 4, GELU activation, a further pointwise projection, and stochastic depth. A 2×2 convolution reduces spatial resolution and increases channel depth at every stage. Stage 1 is comprised of 3 blocks (56×56×96), Stage 2 is comprised of 3 blocks (28×28×192), Stage 3 is comprised of 9 blocks (14×14×384), and Stage 4 is comprised of 3 blocks (7×7×768). Global average pooling reduces spatial dimension after the final stage, followed by LayerNorm and a fully connected layer for classification. ConvNeXt uses only standard convolutions but incorporates transformer inspired design principles like large kernel convs and LayerNorm with high accuracy and vision task scalability.

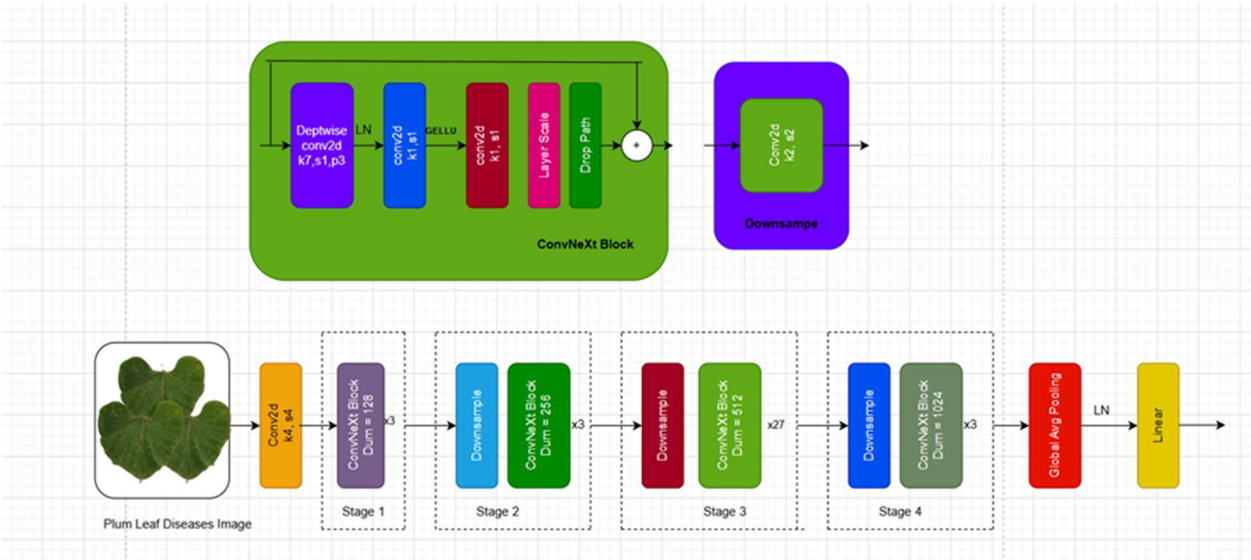


**Fig-4:** Model 1 (ConvNeXt) design architecture

### 1.2.2 ResNext-50

The picture shows ResNeXt-50 model design, a type of convolutional neural network and an extension to ResNext adding a split-transform-merge group convolution pattern to the network. The network initiates with max pooling and 7×7 convolution, shrinking the input but expanding channels. It then traverses four foundational stages, multiple blocks of series. Each bottleneck block is composed of three layers: a 1×1 dimension-reduction convolution, a 3×3 grouped convolution (32 groups for ResNeXt-50) for parallel feature transformation, and another 1×1

dimension-restoring convolution. A shortcut connection adds the block's input and output. Stage 1 consists of 3 blocks of 256 output channels; Stage 2 consists of 4 blocks of 512

channels, Stage 3 consists of 6 blocks of 1024 channels, and Stage 4 consists of 3 blocks of 2048 channels. Down sampling is performed between the stages using convolutions with stride 2. A global average pooling layer reduces the spatial dimensions at the end, and a fully connected layer provides class predictions. The grouped convolution is the key innovation of ResNeXt and allows for greater model capacity without commensurate increase in computational cost. This structure allows ResNeXt-50 to have high accuracy while being efficiently computed.

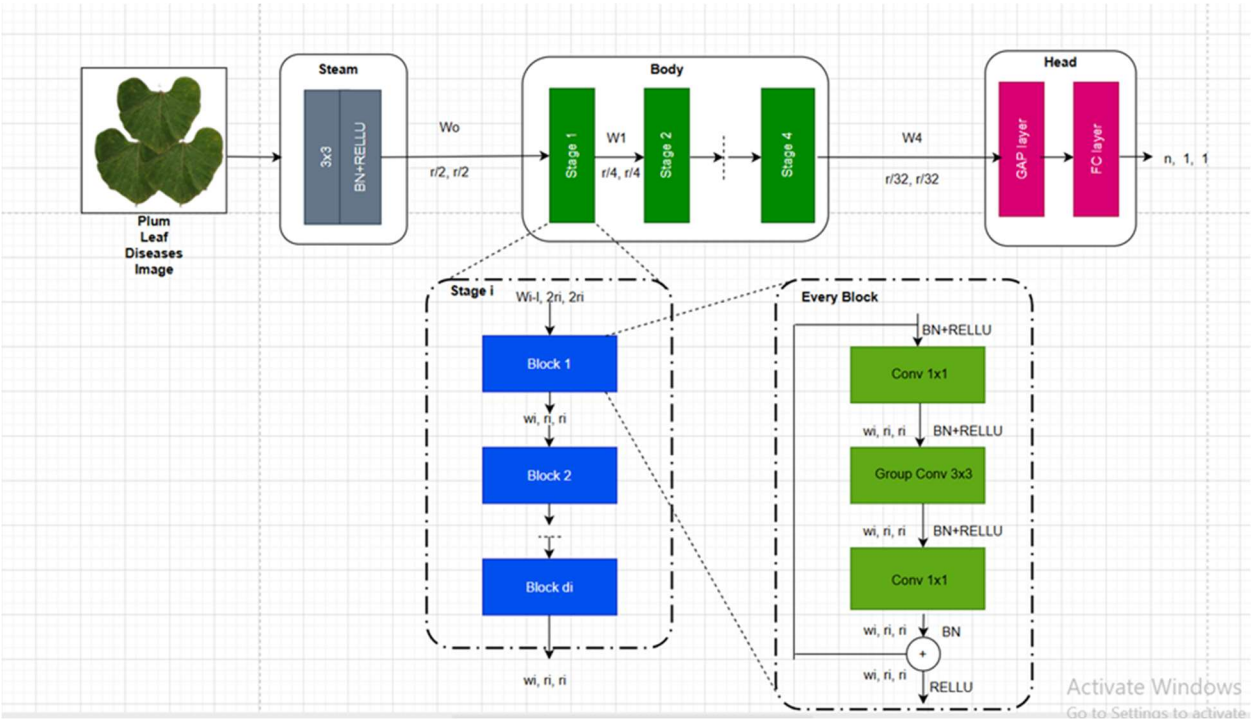


**Fig-5:** Model 2 (ResNeXt-50) design architecture

### 1.2.3 DenseNet-201

DenseNet-201 is a deep convolutional neural network that focuses on enhancing feature propagation and reducing the number of parameters compared to traditional architectures. The network begins with an initial convolutional layer with a large 7x7 filter and stride 2, followed by a max pooling layer that down samples the spatial dimensions. The architecture's backbone consists of four dense blocks, each containing multiple convolutional layers. DenseNet's novelty is in its dense connectivity: every layer in a dense block takes the feature maps of all layers that come before it as additional inputs. This design encourages feature reuse and alleviates gradient flow within the network, enabling more effective training. Between the dense blocks are transition layers, which shrink the model by applying a 1x1 convolution and then a 2x2 average pooling

operation to halve the number of feature maps and maintain computational efficiency. After the final dense block, a global average pooling layer combines the feature maps, and a fully connected layer with a SoftMax activation function produces the final classification output. DenseNet-201, comprised of 201 layers, is a deep and parameter- efficient and hence suitable for complex image classification tasks at comparatively low computational cost.

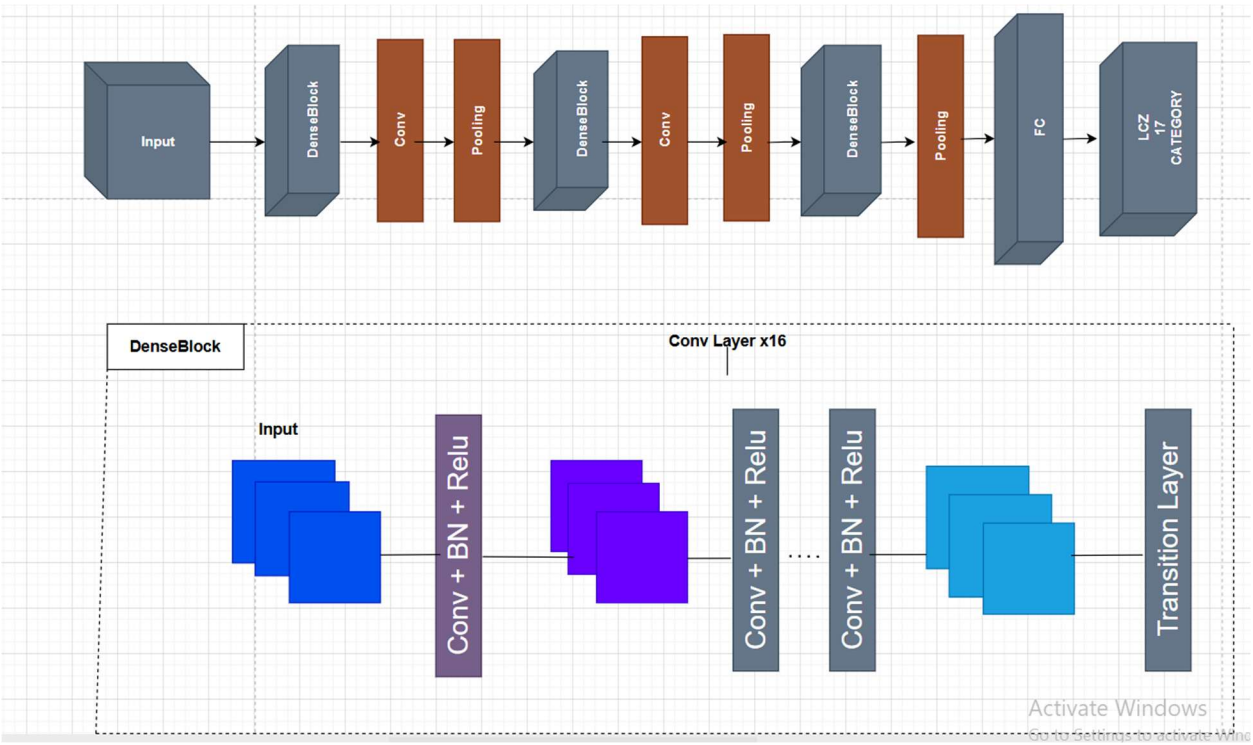


**Fig-6:** Model 4 (DenseNet-201) design architecture

### 1.2.4 EfficientNetV2-S

EfficientNetV2-S is a compact and efficient convolutional neural network that has been optimized for speed and accuracy in image classification. The architecture begins with a standard convolutional stem that reduces the size of the input image while expanding the number of channels. The network is essentially built from a sequence of mobile inverted bottleneck convolution (MBConv) blocks, which are efficient building blocks made of depth wise separable convolutions with squeeze-and-excitation modules to improve feature representation. The blocks are of varied expansion ratios, kernel sizes, and strides to progressively extract features and reduce spatial dimensions. In early network stages, smaller kernel sizes and fewer layers are used to maintain computational efficiency. In deeper networks, the more complex operations are deployed to identify higher-level semantic features. One of the advances in EfficientNetV2 is utilizing fused MBConv layers earlier, combining expansion as well as depth wise convolution into a single step for improved speed on modern hardware. The model concludes with a final convolutional layer,

followed by a global average pooling layer and a fully connected output layer for classification. EfficientNetV2-S balances model size, speed, and accuracy fairly well, making it suitable for real-time execution on low-resource devices.

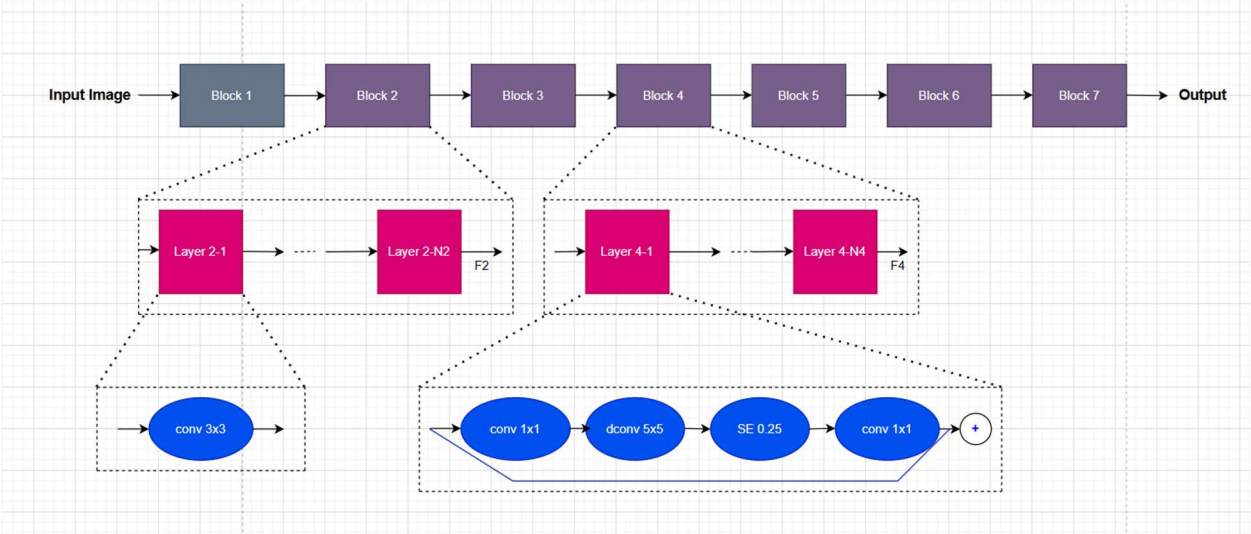


**Fig-7:** Model 5 (EfficientNetV2-S) design architecture

### 1.2.5 NASNet-Large

NASNet-Large is an advanced deep convolutional neural network designed through neural architecture search, which helps in automatically identifying the most effective structure. It primarily consists of two types of blocks: reduction cells and normal cells. Normal cells maintain the spatial dimensions of the input feature maps, while reduction cells decrease the resolution and boost the number of channels. The network starts with a stem block that incorporates convolution and pooling layers, laying the groundwork for more complex processing. After that, it stacks a combination of normal and reduction cells to capture features at different levels. These cells are repeated at various points in the network, enabling it to scale efficiently without losing accuracy. At the end, global average pooling condenses the spatial resolution of the feature maps into a single vector, which is then processed by a fully connected layer followed by SoftMax activation for classification. NASNet-Large shines in image classification tasks by finetuning both the structure of the cells and their arrangement. It skillfully manages depth, width, and resolution, making it a formidable choice for tackling ImageNet-sized image datasets.

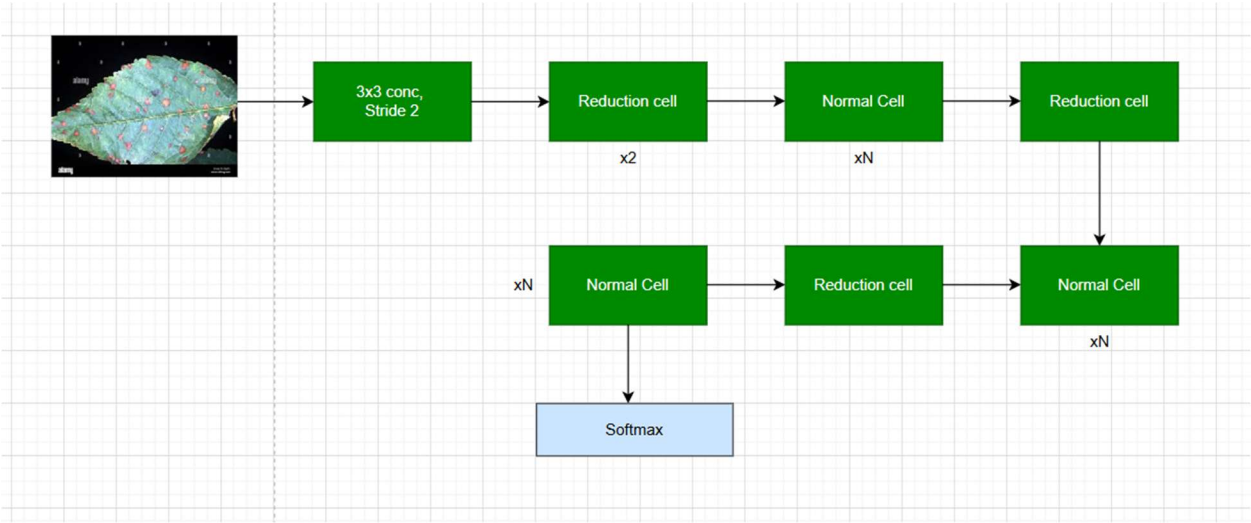


**Fig-8:** Model 6 (NASNet-Large) design architecture

## 2. Result Analysis

**Table 3:** Performance of Experimental Models on Dataset Using Accuracy, Precision, Recall, and F1-Score

| **Metric** | **ConvNeXt** | **RegNet** | **ResNeXt-50** | **DenseNet-201** | **EfficientNetV2-S** | **NASNet-Large** |
|---|---|---|---|---|---|---|
| Accuracy | 95.95% | 95.24% | 95.68% | 95.51% | 95.51% | 96.74% |
| Precision | 96.13% | 95.77% | 95.73% | 95.78% | 95.61% | 96.80% |
| Recall | 95.95% | 95.24% | 95.68% | 95.51% | 95.51% | 96.74% |
| F1-score | 95.99% | 95.35% | 95.70% | 95.57% | 95.34% | 96.76% |

We used six models across three datasets for this project. For Dataset, we employed the following models: **ConvNeXt**, **RegNet**, **ResNeXt-50**, **DenseNet201**, **EfficientNetV2-S**, and **NASNet-Large**. For the ConvNeXt model, the Accuracy, Precision, Recall, and F1-score were 95.95%, 96.13%, 95.95%, and 95.99%, respectively. For **RegNet**, the corresponding values were 95.24%, 95.77%, 95.24%, and 95.35%. The **ResNeXt-50** model achieved 95.68%, 95.73%, 95.68%, and 95.70%. For **DenseNet-201**, the results were 95.51%, 95.78%, 95.51%, and 95.57%. The **EfficientNetV2** s model produced 95.51%, 95.61%, 95.51%, and 95.34%. Finally, **NASNet-Large** achieved the highest results with 96.74%, 96.80%, 96.74%, and 96.76%.

**Table 4:** Classification report of the proposed model (ConvNeXt) on Dataset

| **Class** | **AUC** | **Precision** | **Recall** | **F1-score** |
|---|---|---|---|---|
| Healthy | 98.61% | 98.53% | 95.94% | 97.22% |
| Not healthy | 98.61% | 89.34% | 95.96% | 92.53% |

From Tables 4, we observe the classification report of the proposed model for the dataset, all evaluated using the ConvNeXt architecture.

In Dataset, there are two classes that exhibit the same AUC score of 98.61%. For the healthy class, the precision, recall, and F1-score are 98.53%, 95.94%, and 97.21%, respectively. For the not healthy class, the corresponding scores are 89.34% for precision, 95.96% for recall, and 92.53% for F1-score.

## 2.1 Confusion Matrix of the Proposed ConvNeXt Model on the Dataset

This is a confusion matrix for a classification model in the ConvNeXt architecture in the dataset. A confusion matrix is used to validate the performance of a classification model by showing how many times each class was predicted correctly or incorrectly. From the Dataset where class 0 and class 1 are the two target classes. Out of all examples, the model rightly classified 804 samples as class 0 and 285 samples as class 1, Accuracy prediction for class 0 is 96% and class 1 is 90%, which demonstrates good overall performance. However, 34 cases of class 0 were wrongly classified as class 1 (false positives), and 12 cases of class 1 were wrongly classified as class 0 (false negatives). These results show that the model has a high true negative rate and a quite good true positive rate, with minimal misclassifications. The balance between low false negatives and false positives suggests that the model is both specific and sensitive and is doing a good job of discriminating between the two classes with high accuracy.

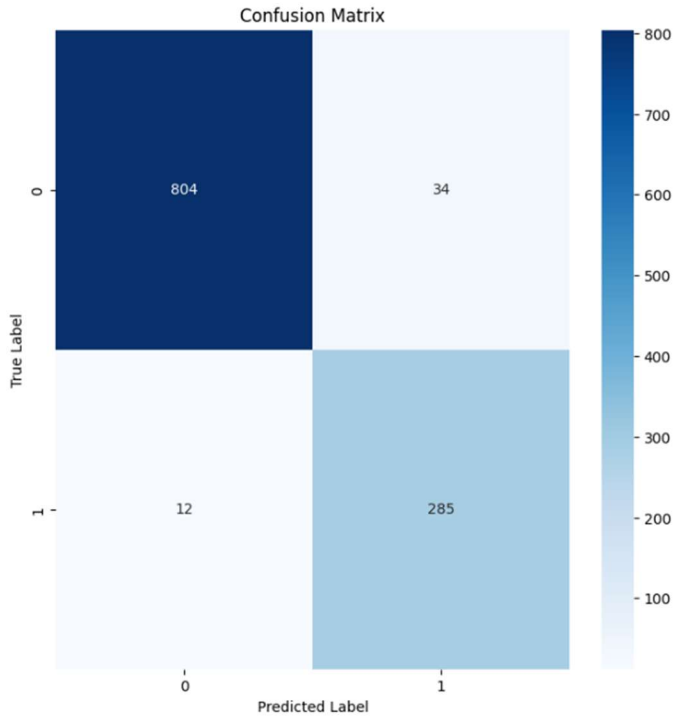


**Fig-9:** Confusion matrix

## 2.2 Learning curves of the proposed model (ConvNext)

The graphs of learning curves for the dataset show the model's behavior over 13 epochs. In the loss curve, both training and validation loss drop sharply in the first few epochs, indicating that the model learns fast. After epoch 3, the training loss maintains a steady decrease to almost 0.05, whereas the validation loss simply plateaus and slightly fluctuates between 0.11 and 0.14. This shows that the model has converged. The widening gap between training and validation losses also shows a bit of over fitting—the model keeps on improving on training data without any more improvement in the validation data.

Supporting the above impression, the accuracy curve also starts with an explosive growth, reaching above 90% within the first few epochs for both training and validation settings. From around epoch 6, training accuracy reaches close to 98%, while validation accuracy stays around 96%. This tiny but persistent gap suggests some degree of over-fitting, but this over-fitting is kept well under control and the model works very well in general.

Thus, the curves confirm the fact that the model learns well and its performance is very good concerning the training and validation set. Techniques like early stopping (7th to 9th epoch), dropout, weight decay, or data augmentation could help in reducing overfitting and thereby enhancing and speeding up generalization beyond this point.

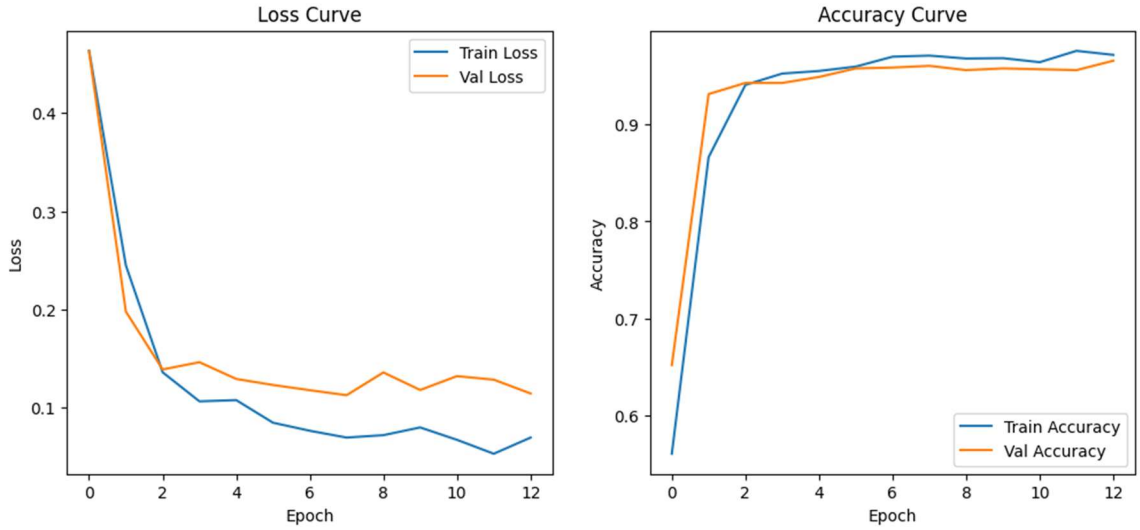


**Fig-10:** Learning Curve

## 2.3 ROC-AUC curves of the proposed model (ConvNeXt)

We Know the ROC-AUC curve is a performance evaluation metric which is commonly used for binary classification problems. Here in the database, we know there are two classes so their ROC-AUC curves are 98.61% for both classes which is nearly 1. This is a very high AUC value, which

suggests that the classifier is very good with an enormously high true positive rate and a very low false positive rate for various threshold values. Typically, the ROC curve and AUC value confirm that the model performs very well to distinguish between the two classes.

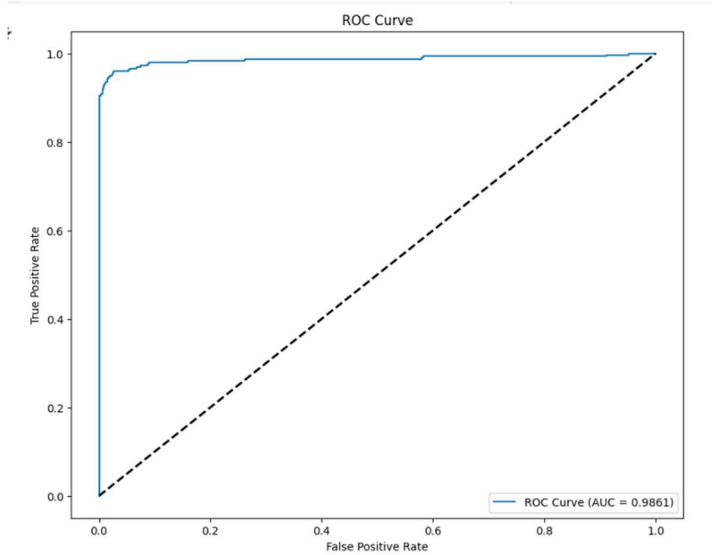


**Fig-11:** ROC-AUC Curve

## 2.4 Grad-CAM Visualization Analysis for the Dataset

From the Dataset we see there are two classes which are healthy (Class 0) and not healthy (Class 1). Grad-CAM visualization illustrates that the deep learning model is able to comprehend and differentiate between healthy (Class 0) and unhealthy (Class 1) leaves by highlighting the most pertinent locations in its decision-making. On the top row, the original image of a healthy leaf is shown with its Grad-CAM heatmap, where the model is considering the smooth, intact, and uniformly green middle section, implying that these features are accountable for it being labeled as healthy. In the lower row, the real image of an unhealthy leaf also has visible damage such as holes and irregular edges, and its corresponding Grad-CAM heatmap shows these specific damaged regions. This would mean that the model is accurately picking up on and using signs of physical damage to classify the leaf as not healthy. Overall, the Grad-CAM outputs confirm that the model relies on substantial and understandable visual information, enhancing the clarity and credibility of its classifications.

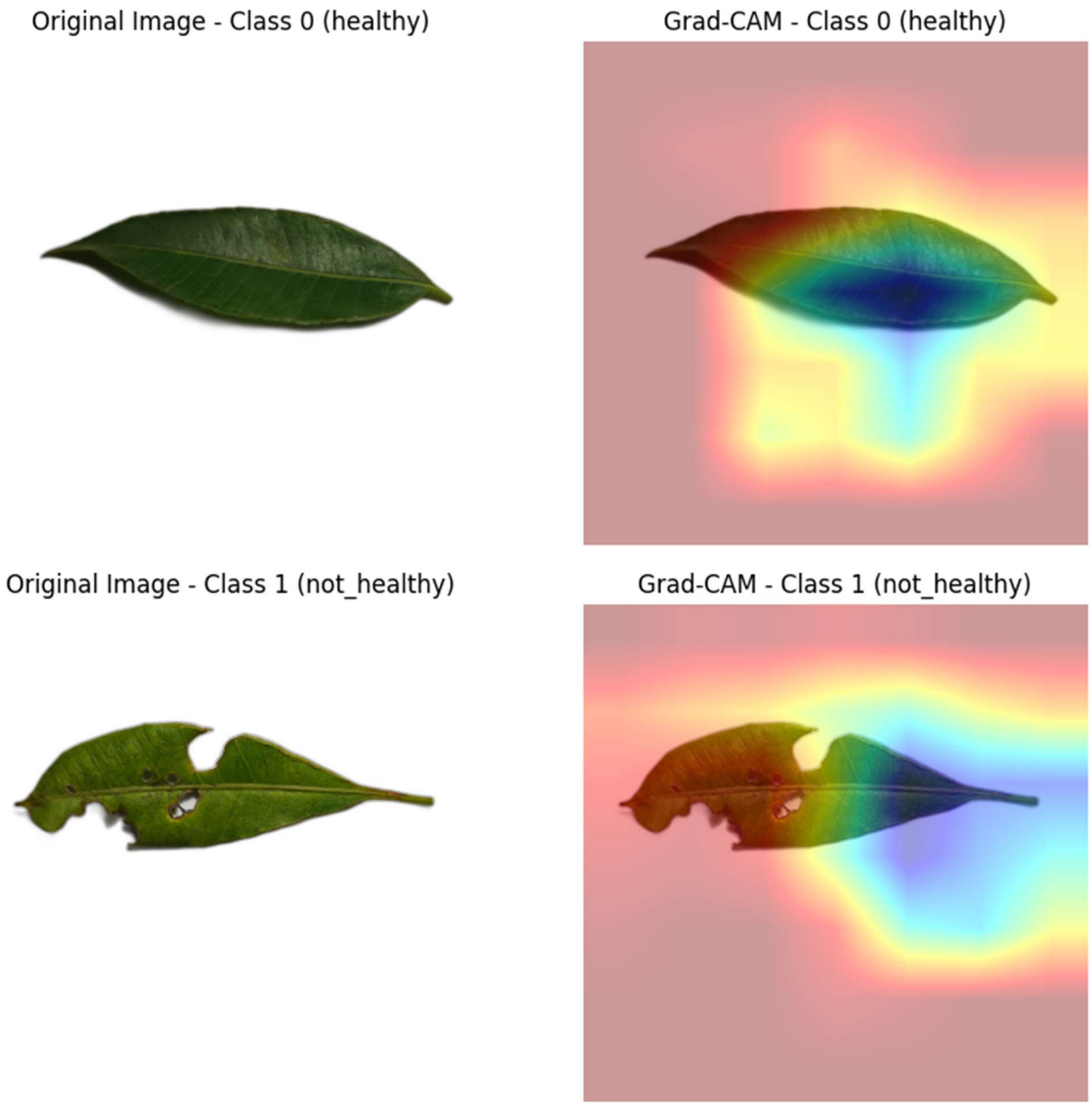


**Fig-12:** Grad-CAM

## Conclusion:

This work presents a deep learning framework for classifying Hog Plum leaf diseases with high accuracy. Among six tested models, NASNet-Large performed best, while ConvNeXt offered a strong balance of accuracy and efficiency. Grad-CAM visualizations confirmed the model's interpretability. The results demonstrate the framework's effectiveness and its potential for real-world agricultural applications in disease detection.